Environment-Grounded Self-Reward for Dense Feedback in RLVR

NeurIPS 2026

Rohan Sikand · Stanford University

2026

Motivation

The Sparse Reward Bottleneck in RLVR

RLVR (GRPO) trains LLMs using **binary pass/fail** from deterministic verifiers.

Two failure modes of binary GRPO advantages $\hat{A}_i = (r_i - \mu)/\sigma$:

1. **All-pass or all-fail groups**: $\sigma = 0 \rightarrow$ zero gradient signal
2. **No credit assignment**: $9/10$ correct $=$ same reward as random nonsense

Meanwhile, the environment produces **rich execution feedback** at every step — stack traces, compiler errors, partial test outputs — **all discarded** after computing pass/fail.

Can we recover this discarded signal as dense reward?

Prior Approaches Fall Short

Approach	Dense?	Fundamental Issue
Process reward models	✓ Step-level	Expensive labels; static; domain-specific
External LLM judge	✓ Flexible	Distributional gap $\rightarrow$ evasion
Self-rewarding LMs	✓ On-policy	Reward collapse (no grounding)
EGSR (ours)	✓ On-policy	Grounded in env feedback

Our insight: Combine the on-policy property of self-reward with objective environment grounding — using information that is already produced and currently discarded.

Method

EGSR: The Policy as Its Own Reward Model

Same model π_θ, two roles, differing only in input context:

$$\underbrace{\pi_\theta(\cdot \mid s)}_{\text{Student: generates rollout}} \qquad \underbrace{\pi_\theta(\cdot \mid s, c_i, p_{\text{judge}})}_{\text{Teacher: judges rollout}}$$

where $c_i = (a_i, f_i, r_i)$ contains the solution, **execution feedback**, and reasoning trace.

Key properties:

- **Parameter-aligned** — zero distributional gap
- **Grounded** — env feedback prevents collapse
- **Cheap** — one extra forward pass

Not "on-policy" — precise:

- Same weights (*capability* aligned)
- Teacher sees privileged info (f_i)
- Like grading your own exam *with the answer key*

Dense Reward via Structured Judgment

Teacher outputs judgment over classes $\mathcal{K}$:

$$R_{\text{self}}(s, a_i) = \sum_{k \in \mathcal{K}} w_k \cdot P_\theta(q = k \mid s, c_i, p_{\text{judge}})$$

Class	Weight	Semantics
`correct`	$+1.0$	Passes all tests
`minor_bug`	$+0.6$	Right approach, small error
`wrong_approach`	$+0.2$	Fundamentally incorrect strategy
`critical_error`	0.0	Crash, timeout, no output
`exploit`	-0.5	Hardcoded / pattern-matched

Result: $R_{\text{self}} \in [-0.5, 1.0]$ — 90%-correct code gets ≈ 0.8, not binary 0.

Label-Free Judge Maintenance

Problem: RL optimizes θ for generation. Nothing explicitly trains the teacher.

Two self-supervised objectives using only signals already in the loop:

1. **Feedback prediction** — predict f_i from (s, a_i):

$$\mathcal{L}_{\text{pred}}(\theta) = -\mathbb{E}\left[\frac{1}{|f_i|}\sum_k \log \pi_\theta(f_{i,k} \mid s, a_i, f_{i,<k})\right]$$

{→ Forces internalization of code → execution causality. Always on-distribution.}

2. **Verifier calibration** — anchor R_{self} to pass/fail:

$$\mathcal{L}_{\text{cal}}(\theta) = \mathbb{E}\Big[(R_{\text{self}}(s, a_i) - R_{\text{env}}(s, a_i))^2\Big]$$

{→ Prevents systematic leniency/harshness. Free.}

Joint: $\theta \leftarrow \theta - \eta\, \nabla_\theta[\mathcal{L}_{\text{GRPO}} + \lambda_1\, \mathcal{L}_{\text{pred}} + \lambda_2\, \mathcal{L}_{\text{cal}}]$

Self-Judge Hacking Defense

Can the policy learn to hack its own judge?

Three lines of defense:

1. $\mathcal{L}_{\text{cal}}$ **prevents global miscalibration** — systematic leniency directly increases calibration loss
2. $\mathcal{L}_{\text{pred}}$ **constrains the internal execution model** — accurate feedback prediction makes lenient judgment on those same error types *inconsistent* with shared representations
3. **Consistency monitoring** — track correlation between R_{self} and error-severity heuristics over training

Residual risk: *local* leniency on specific failure types while aggregate stays calibrated. We measure this in Experiment 4.

Theory

Theoretical Motivation

Proposition 1: Gradient variance reduction

- Binary: $\mathrm{Var}[\hat{A}] \propto p(1-p)/G \to$ **vanishes** at extreme pass rates
- Dense: $\mathrm{Var}[\hat{A}] \geq \sigma_R^2/G > 0$ **regardless** of p
- $\to$ Dense reward guarantees signal in exactly the regimes where binary fails

Proposition 2: Information-theoretic bound

- $I(Q^*; f \mid R_{\mathrm{env}}, a) > 0$ — feedback resolves ambiguity about *why* a rollout failed
- Upper bounds improvement from any method leveraging f_i

Proposition 3: Calibration safety

- If $\mathcal{L}_{\mathrm{cal}} = 0$: optimal policies under $R_{\mathrm{self}} \subseteq$ optimal policies under R_{env}
- $\to$ Dense reward doesn't introduce false optima

Experiments

Setup

Framework: verl + FSDP2 + vLLM

Models: Qwen3-4B (primary) · Qwen3-14B (scale) · OLMo3-7B (secondary)

Domains: Code (LeetCode, LiveCodeBench) · Math (MATH, AIME)

Method	**Dense?**	**Adapts?**	**Labels?**	**Reward**
GRPO (binary)	✗	N/A	No	$R_{\text{env}} \in \{0, 1\}$
Test-case fraction	✓	N/A	No	$R = k/n$
External judge (Haiku)	✓	Static	No	LLM score
Ungrounded self-reward	✓	On-policy	No	No f_i
EGSR + $\mathcal{L}_{\text{CE}}$	✓	On-policy	Yes	Labeled buffer
EGSR (label-free)	✓	On-policy	**No**	$\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$

Signal Validation (E1)

Frozen Qwen3-4B, 5K rollouts, no RL. Teacher AUC under context ablations:

Context	AUC	Cal. Error
(a) Solution only	**[X.XX]**	**[X.XX]**
(b) + exec feedback f_i	**[X.XX]**	**[X.XX]**
(c) + reasoning trace r_i	**[X.XX]**	**[X.XX]**
(d) Full context	**[X.XX]**	**[X.XX]**

[Describe: (b) significantly above (a) confirms execution feedback is the key ingredient. Discuss per-class accuracy.]

Information estimate: $I(Q^*; f \mid R_{\text{env}} = 0, a) =$ **[X.XX]** bits

Main Results (E2)

Method	LeetCode	LiveCodeBench	MATH	AIME
GRPO (binary)	[XX.X]	[XX.X]	[XX.X]	[XX.X]
Test-case fraction	[XX.X]	[XX.X]	[XX.X]	[XX.X]
External judge (Haiku)	[XX.X]	[XX.X]	[XX.X]	[XX.X]
Ungrounded self-reward	[XX.X]	[XX.X]	[XX.X]	[XX.X]
EGSR + $\mathcal{L}_{\text{CE}}$	[XX.X]	[XX.X]	[XX.X]	[XX.X]
EGSR (label-free)	[XX.X]	[XX.X]	[XX.X]	[XX.X]

[Key finding: EGSR improves Pass@1 by X% over binary GRPO, matches external judge at Y$\times$ lower cost.]

Convergence and Gradient Analysis

Convergence curves

[Figure: Pass@1 vs training steps. EGSR converges faster and higher than binary GRPO.]

Steps to target: **[EGSR reaches X% at step Y vs step Z for binary GRPO]**

Gradient norm by pass rate

[Figure: Gradient norm stratified by p. Binary GRPO collapses at extreme p; EGSR maintains signal.]

Validates Proposition 1.

Stratified Analysis

Method	Easy	Medium	Hard
GRPO (binary)	**[XX.X]**	**[XX.X]**	**[XX.X]**
EGSR	**[XX.X]**	**[XX.X]**	**[XX.X]**
Δ	**[+X.X]**	**[+X.X]**	**[+X.X]**

[Expected: largest gains on medium difficulty — diverse failure modes, mixed pass rates.]

Within-bucket analysis: Among rollouts with identical k/n pass fraction, EGSR assigns different R_{self} that better predicts **[edit distance / full-suite pass rate]**.

$\rightarrow$ Self-judgment adds signal beyond counting.

Judge Maintenance (E3) — Key Novel Result

Teacher AUC tracked every 20 steps throughout RL training:

[Figure: AUC vs training step for 5 conditions: (a) no maintenance — degrades; (b) $\mathcal{L}_{\text{pred}}$ only; (c) $\mathcal{L}_{\text{cal}}$ only; (d) $\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$ — stays flat; (e) labeled buffer — degrades slowly.]

Key findings:

- **[(a) degrades by X AUC]**
- **[(e) degrades by Y AUC (stale)]**
- **[(d) maintains within Z AUC]**

Implication:

- Label-free $\geq$ labeled
- $\mathcal{L}_{\text{pred}}$ is the key component
- Always on-distribution $>$ static buffer

Reward Hacking and Scale (E4–E5)

E4: Reward hacking dynamics

[Figure: Hack rate over training. Binary GRPO vs EGSR w/o exploit vs EGSR w/ exploit.]

[Describe finding. Exploit class suppresses hacking by X%.]

E5: Scale

	4B	14B
Δ Pass@1	[+X.X]	[+X.X]
Teacher AUC	[X.XX]	[X.XX]

[Does EGSR delta grow with scale?]

Ablations

Judgment granularity

	Pass@1
Binary	**[XX.X]**
5-class	**[XX.X]**

Group size

	G=4	G=8	G=16
GRPO	**[X]**	**[X]**	**[X]**
EGSR	**[X]**	**[X]**	**[X]**

[EGSR G=4 $\approx$ GRPO G=16?]

λ sensitivity

[Heatmap of $\lambda_1 \times \lambda_2$. Stable across order of magnitude?]

Warm-start SFT

	Pass@1
No warm-start	**[XX.X]**
With warm-start	**[XX.X]**

[Can $\mathcal{L}_{\text{pred}}$ bootstrap from scratch?]

Discussion

Limitations and Future Work

Limitations:

- EGSR value scales with **feedback richness** — weaker gains in minimal-feedback domains
- Self-judgment capacity at 4B may be insufficient for fine-grained evaluation **[report]**
- Cannot fully rule out local leniency in self-judge **[report E4]**
- Evaluated on code + math only

Future directions:

- **Non-verifiable domains:** NLI for summarization, format checkers for instruction following — weaker but still grounding
- **Curriculum from self-judgment:** variable R_{self} identifies the learning frontier
- **Step-level self-reward:** extend teacher to judge individual reasoning steps (self-supervised PRM)

Summary

Observation: RLVR environments produce rich feedback that is currently discarded.

Method: EGSR — the policy judges its own rollouts conditioned on execution feedback.

Key results:

- **[X%]** improvement over binary GRPO on **[benchmarks]**
- Matches external LLM judge at **[Y×]** lower cost
- Label-free maintenance sustains teacher quality; labeled buffer degrades
- **[Z×]** compute reduction via smaller group size

Core contributions:

1. Dense, grounded, parameter-aligned self-reward for RLVR
2. Label-free judge maintenance ($\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$)
3. Theoretical guarantees: variance reduction + calibration safety